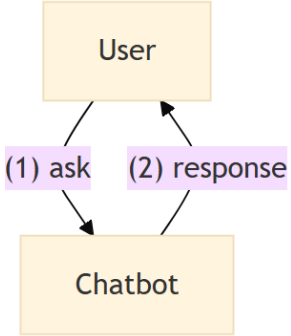
Intelligent modeling

– generative AI–

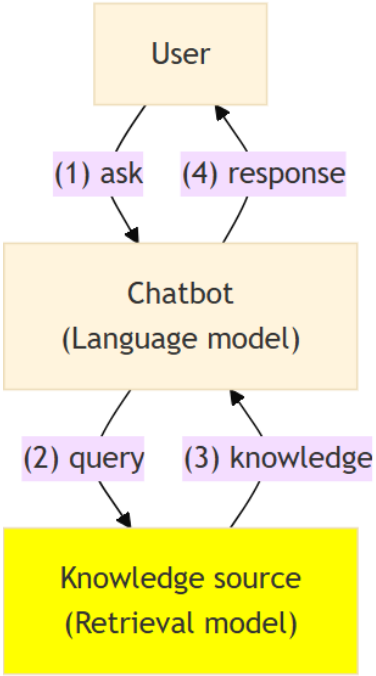
Laura Dioşan

Intelligent systems

- A simple Chatbot

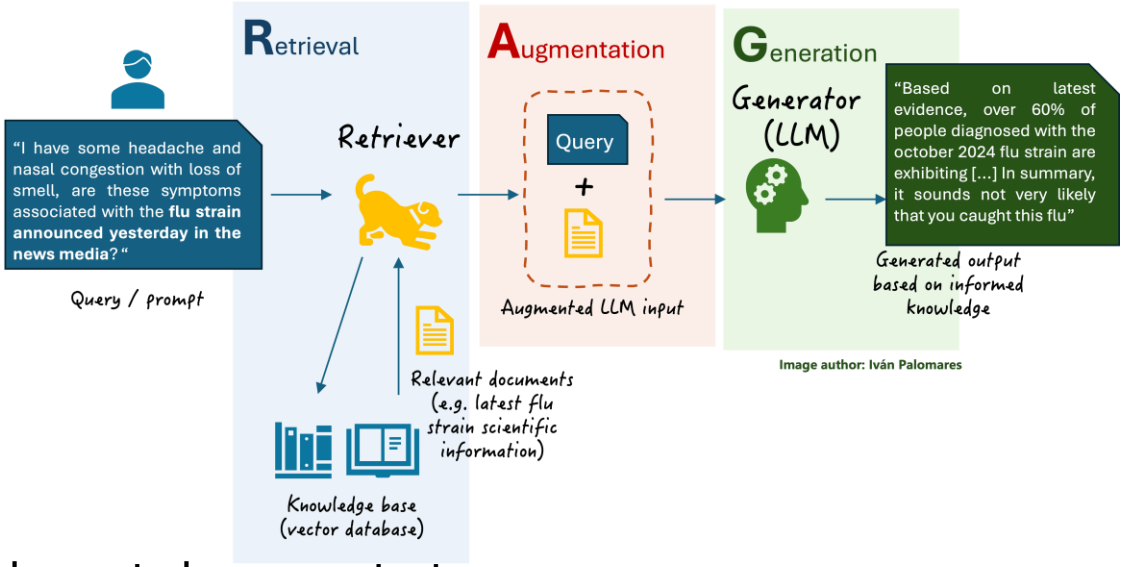


- A chatbot with external Knowledge



Retrieval-Augmented Generation (RAG)

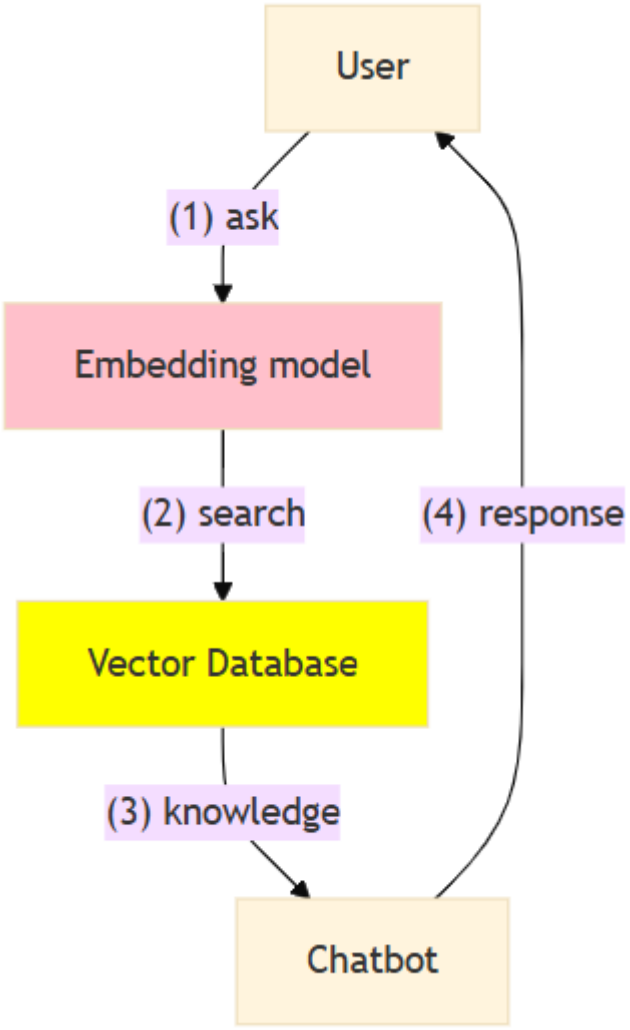
- Aim
  - enhance LLMs by incorporating up-to-date and truthful contextual information in user queries or prompts



- Retriever
  - Finds and retrieves relevant documents to user query
- Augmented query
  - Incorporates contextual knowledge from the retrieved documents
- Generator
  - Retrieves the user query augmented with relevant contextual information
  - Generates a more precise and truthful text response

RAG systems

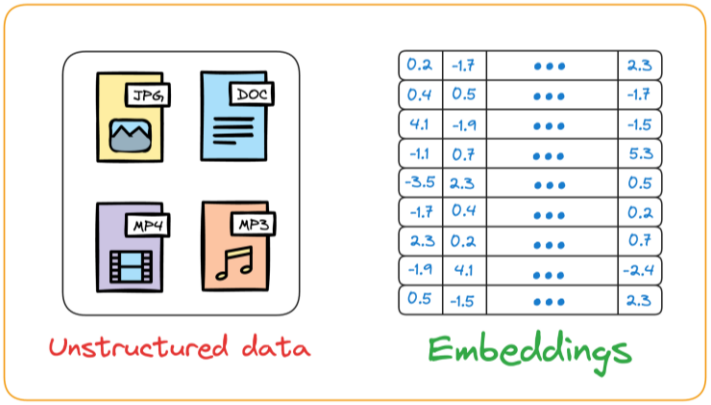
- Components
  - Queries (prompts) formulated by the user in natural language
  - Embedding model
    - A pre-trained language model that converts input text into embeddings
  - A retrieval model
    - fetches relevant information from an external knowledge
  - **Vector database (knowledge base)**
    - A storage system for knowledge and its corresponding embedding vectors
    - E.g. Qdrant, Pinecone, pgvector
  - A language model
    - generates responses based on the retrieved knowledge
    - E.g. Llama, Gemma, or GPT



Vector databases

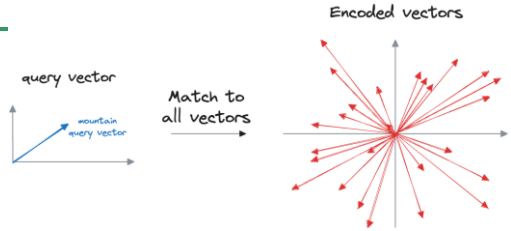
- Vector database - stores unstructured data
  - text, images, audio, video, etc.

  in the form of vector embeddings (high dimensional vectors – HD vectors)

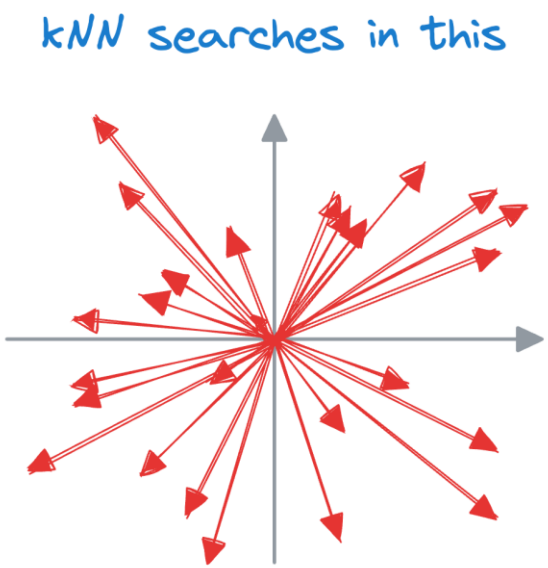


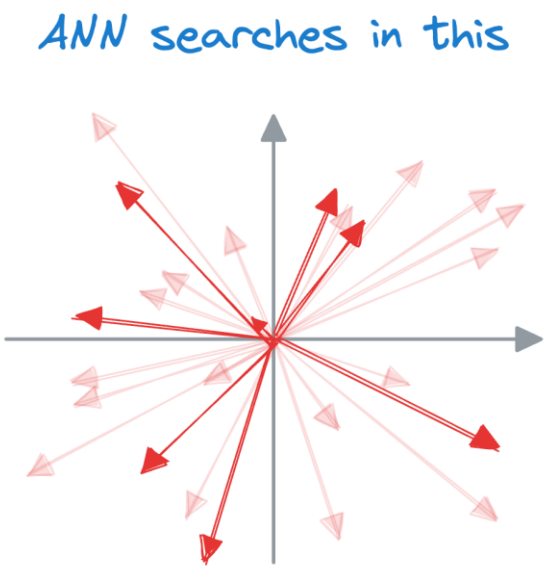
- Database type
  - Traditional DBs
    - Highly efficient for exact matching (e.g. "find a user with ID 12345")
    - Index structure: B-trees optimized for range queries
  - Vector DBs
    - DBs purpose-built for similarity search
    - Efficient for high-dimensional geometry (embeddings)
      - How? By algorithms that exploit the structure of embedding spaces to avoid brute-force distance calculations
      - Trade-offs between
        - search speed, Accuracy, Memory usage

Vector databases



- Searching
  - Exact match
    - Nearest Neighbour (NN) search – too complex
  - Approximative match
    - Approximate NN search

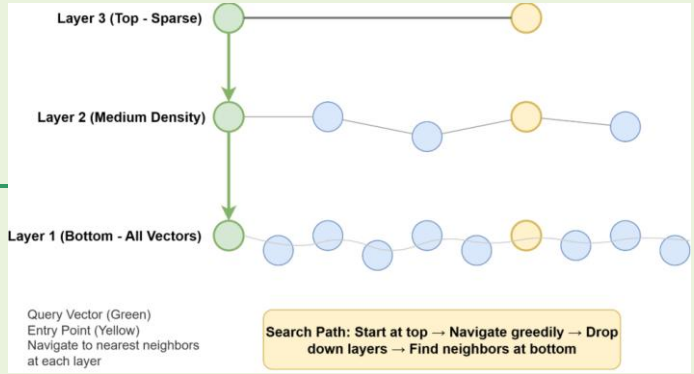




Vector databases

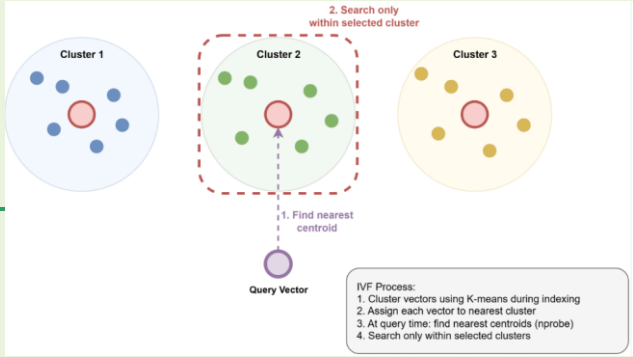
- Index approaches
  - Hierarchical Navigable Small World (HNSW)
  - Inverted File Index (IVF)
  - Product Quantization (PQ)

Vector databases



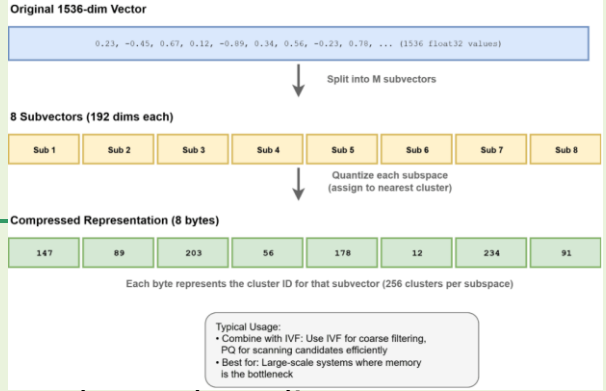
- Index approaches
  - **HNSW**
    - Main idea:
      - builds a multi-layer graph structure - each layer contains a subset of vectors connected by edges
        - The top layer is sparse, containing only a few well-distributed vectors.
        - Each lower layer adds more vectors and connections, with the bottom layer containing all vectors
      - Search starts at the top layer and greedily navigates to the nearest neighbor.
        - Once it can't find anything closer, it moves down a layer and repeats.
        - This continues until reaching the bottom layer, which returns the final nearest neighbors.
    - Complexity
      - Search complexity is O(log N)
    - Advantages
      - excellent recall and speed → Ideal for latency-sensitive apps (real-time fraud detection)
    - Weaknesses
      - requires keeping the entire graph in memory → expensive for massive datasets

Vector databases



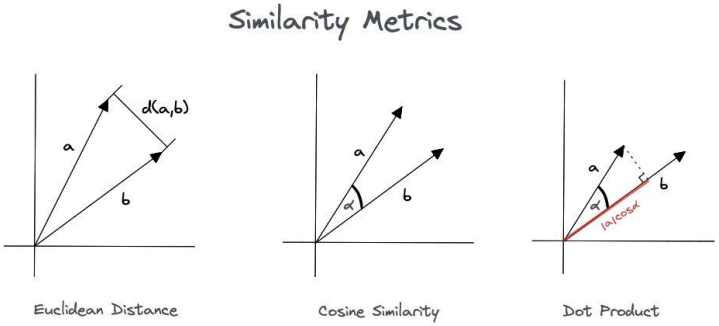
- Index approaches
  - **IVF**
    - Main idea:
      - partitions the vector space into regions using clustering algorithms like K-means.
      - During indexing, each vector is assigned to its nearest cluster centroid.
      - During search, you first identify the most relevant clusters (top-k), then search only within those clusters.
      - uses less memory than HNSW because it only loads relevant clusters during search
    - Complexity
      - centroid search: $O(n_{clusters})$
      - in-cluster search: $O(list_{size})$
      - effectively behaves like O(log N)
    - Advantages
      - scales to very large datasets (for datasets too large for RAM – e-commerce or multimedia app)
      - Lower memory footprint
      - Search speed/accuracy tradeoff is configurable
        - Search more clusters → higher recall
        - Search fewer clusters → faster speed
    - Weaknesses
      - lower recall at the same speed
      - requires careful tuning

Vector databases



- Index approaches
  - **PQ**
    - Main idea:
      - compresses HD vectors to reduce memory usage and speed up distance calculations.
      - splits each vector into subvectors, then clusters each subspace independently.
      - During indexing, each subvector is replaced by the ID of its closest centroid
        - vectors are represented as sequences of cluster IDs
        - huge compression (e.g., 512-D → a few bytes)
      - during search, approximate distances are computed using precomputed lookup tables, enabling extremely fast scanning
      - works best combined with other methods: IVF for initial filtering, PQ for scanning candidates efficiently
    - Complexity
      - effectively behaves like O(log N) when paired with IVF filtering
      - Search complexity:
        - scanning candidates is $O(N / n_{clusters})$ when combined with IVF
        - lookup-table distance computations are O(number of subvectors)
      - Encoding complexity: linear in vector dimensionality
    - Advantages
      - excellent for very large datasets
      - dramatic memory savings (10×–100× compression) → enables disk-based or hybrid storage
      - fast scanning due to LUT-based distance evaluation
    - Weaknesses
      - lower recall at the same speed
      - quantization introduces information loss → distance estimates are approximate

Vector databases



- Similarity measures between HD vectors
  - Euclidean distance
    - core: measures straight-line distance
    - + intuitive
    - - sensitive to vector magnitude (2 HD vector pointing the same direction but with different lengths are considered dissimilar)

$$d_{euclid}(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^{d} (x_i - y_i)^2}$$

  - Cosine similarity
    - core: measures the angle between vectors (ignoring magnitude)
    - + ideal for embeddings where direction encodes meaning, but scale doesn't.
    - - ignores vector magnitude → cannot capture intensity/importance differences
    - - sensitive to very small vector norms (numerical instability)

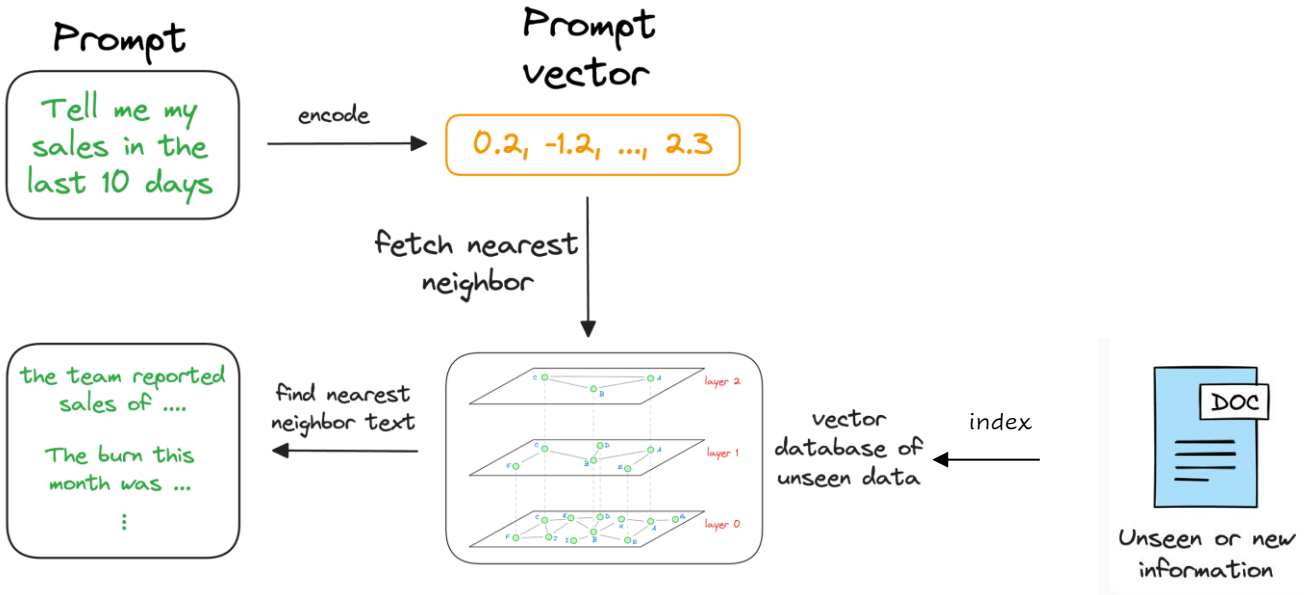
$$\text{cosine_sim}(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \, \|\mathbf{y}\|}$$

  - Dot product
    - core: cosine similarity without normalization
    - + when all vectors are unit length (normalized vectors), it's equivalent to cosine similarity but faster to compute
    - - biased toward vectors with larger magnitude → not meaningful without normalization

$$\text{dot}(\mathbf{x}, \mathbf{y}) = \mathbf{x} \cdot \mathbf{y} = \sum_{i=1}^{d} x_i y_i$$

Vector databases

- LLM flow



Vector databases

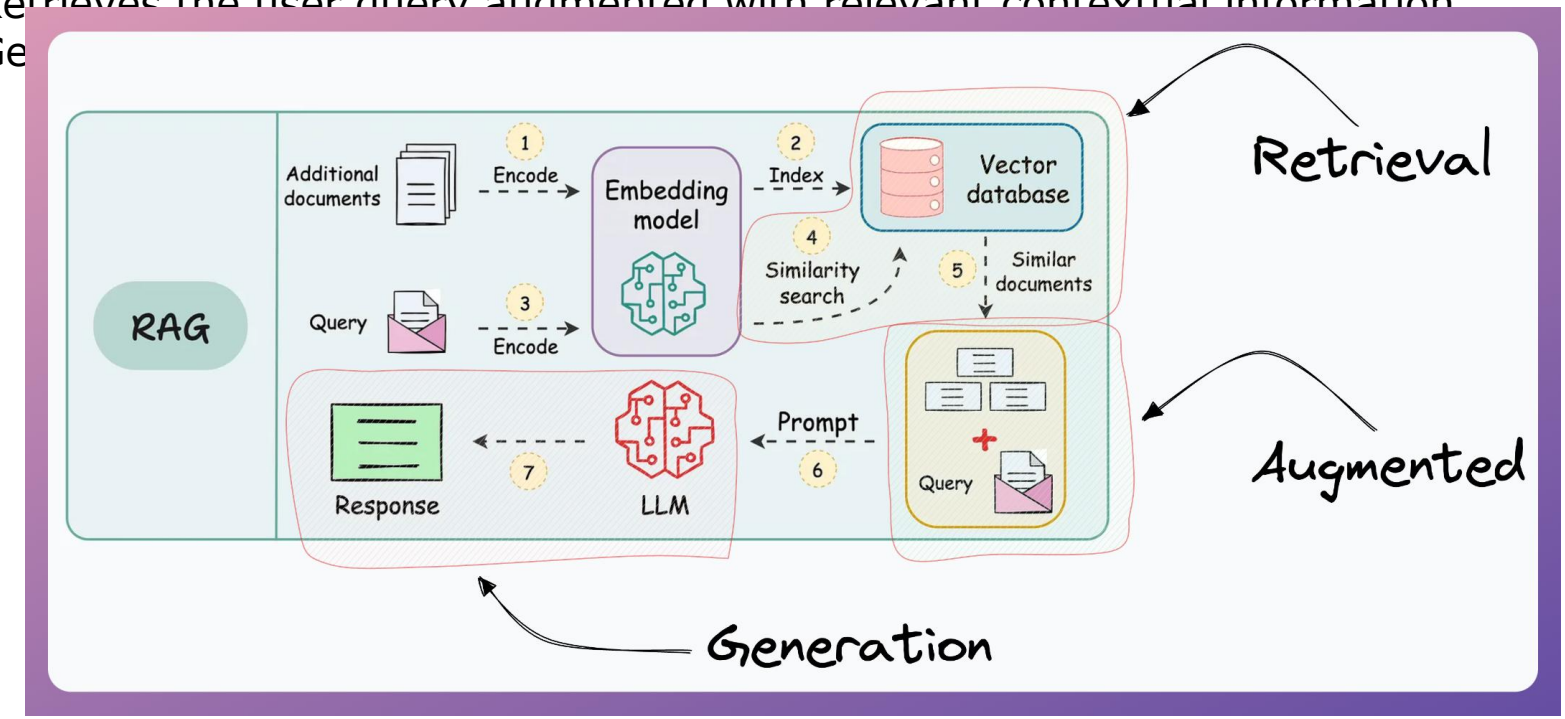
- When to use?
  - No need for a vector databases:
    - fewer than 100K vectors. Brute-force search with NumPy should be fast enough.
    - vectors that change constantly - the indexing overhead might exceed search savings.
    - Need perfect accuracy - use exact search with optimized libraries (e.g. FAISS).
  - Use vector databases:
    - Have millions of vectors and need low-latency search.
    - Are building semantic search, RAG, or recommendation systems at scale.
    - Need to filter vectors by metadata while maintaining search speed.
    - Want infrastructure that handles sharding, replication, and updates.

- Frameworks
  - Pinecone
  - Weaviate
  - Chroma
  - Qdrant
  - Milvus
  - Postgres with pgvector
  - ElasticSearch

Retrieval-Augmented Generation (RAG)

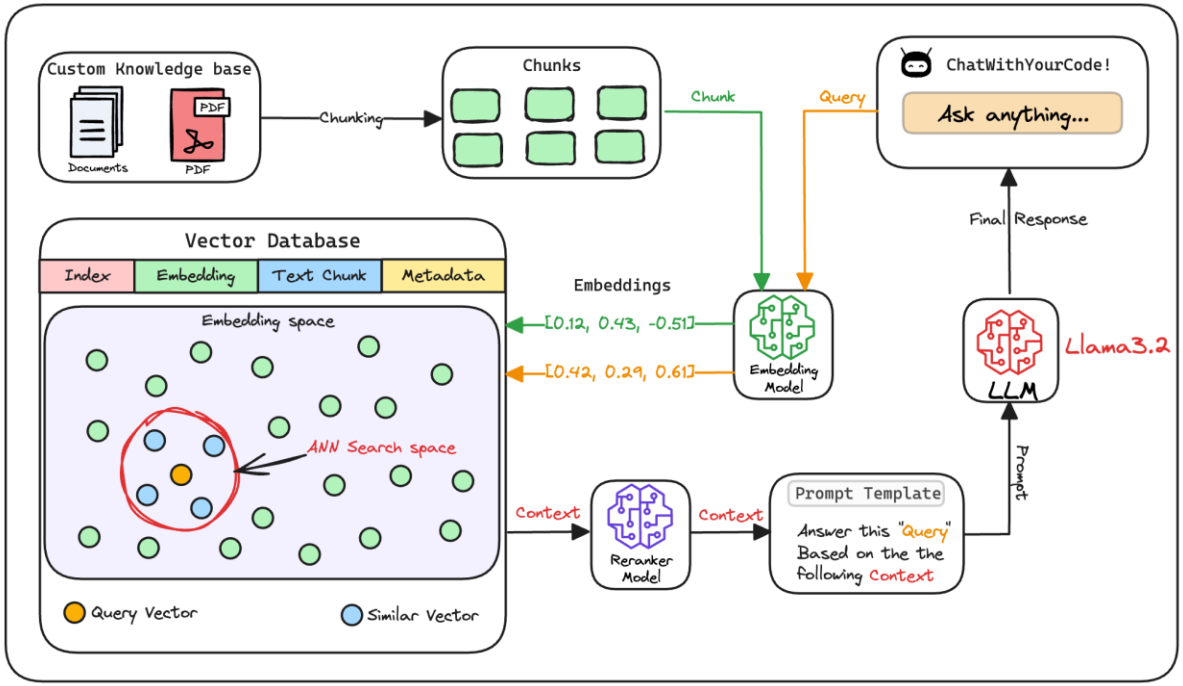
- Retrieval: Accessing and retrieving information from a knowledge source, such as a database or memory.
- Augmented: Enhancing or enriching something, in this case, the text generation process, with additional information or context.
- Generation: The process of creating or producing something, in this context, generating text or language.

- Retriever
  - Finds and retrieves relevant documents to user query
- Augmented query
  - Incorporates contextual knowledge from the retrieved documents
- Generator
  - Retrieves the user query augmented with relevant contextual information
  - Ge



RAG systems

- Workflow

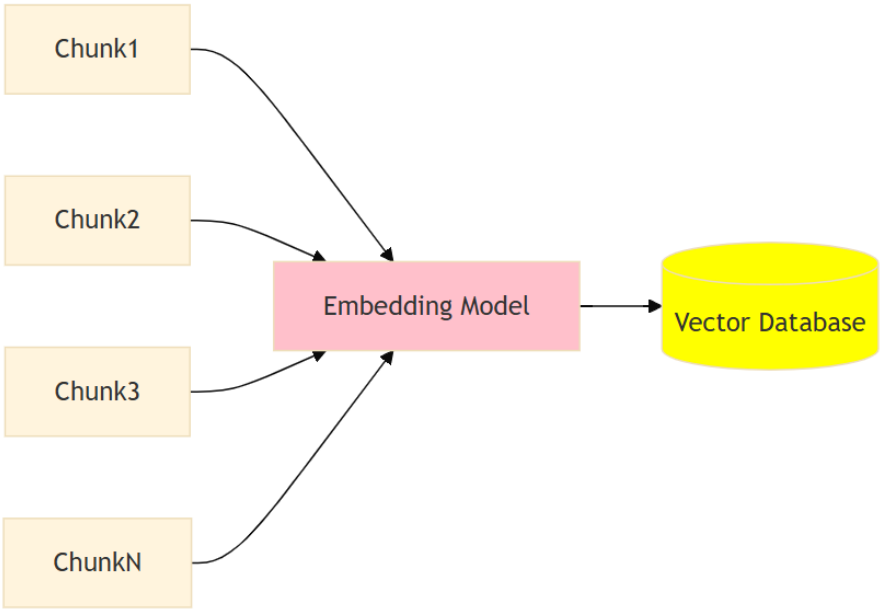


RAG

- Workflow
  - Indexing phase
    - Chunking
    - Similarity
  - Retrieval phase

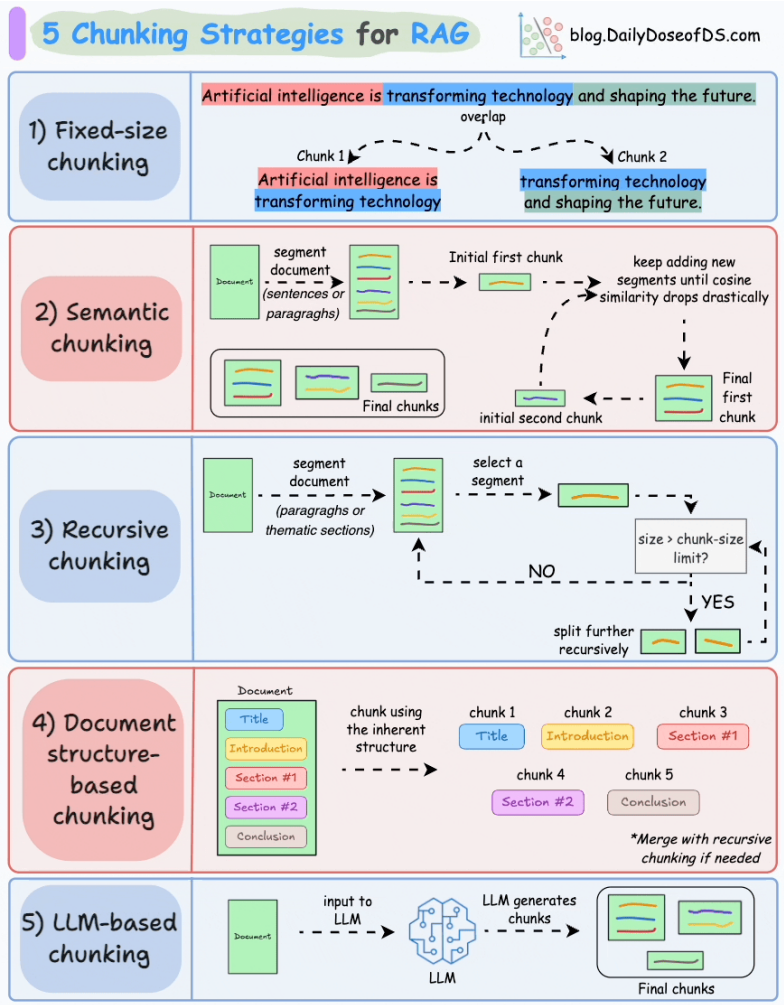
RAG

- Workflow
  - Indexing phase
    - breaking the dataset (or documents) into small **chunks** and calculating a vector representation for each chunk



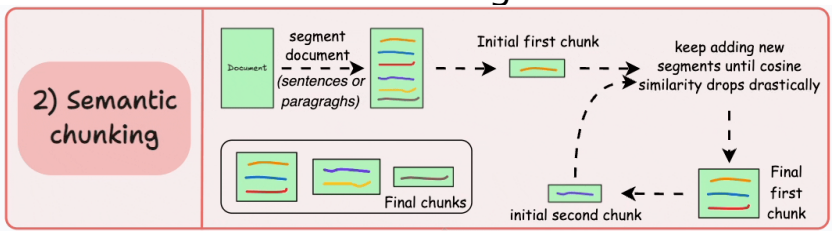
RAG -workflow

- Indexing phase
  - Chunking

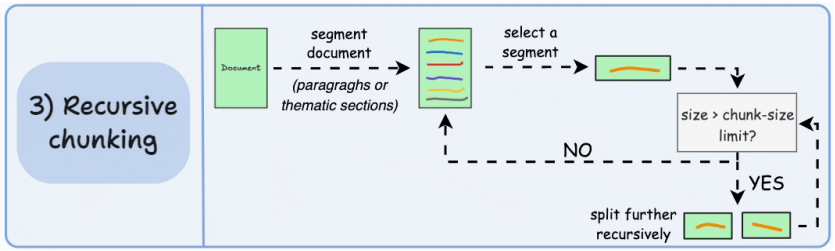


RAG -workflow

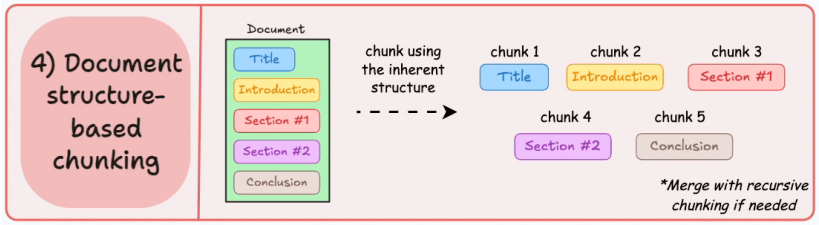
- Chunking
  - Fixed-size chunking
  - Semantic chunking



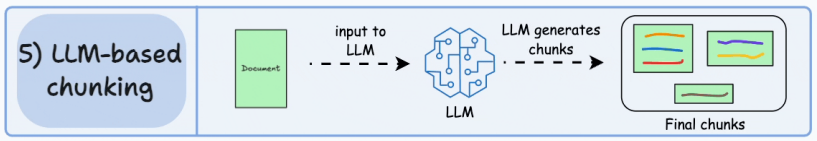
  - Recursive chunking

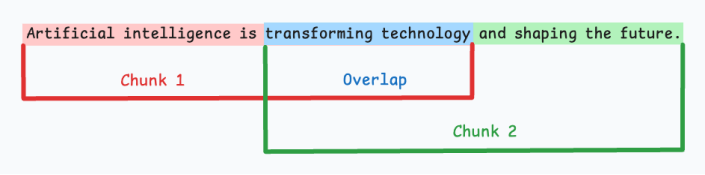


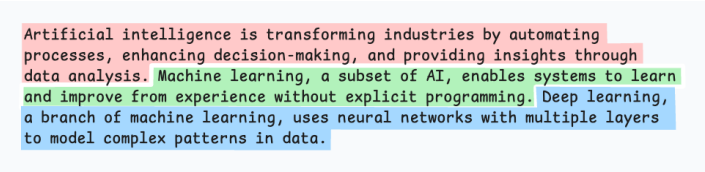
  - Document structure-based chunking

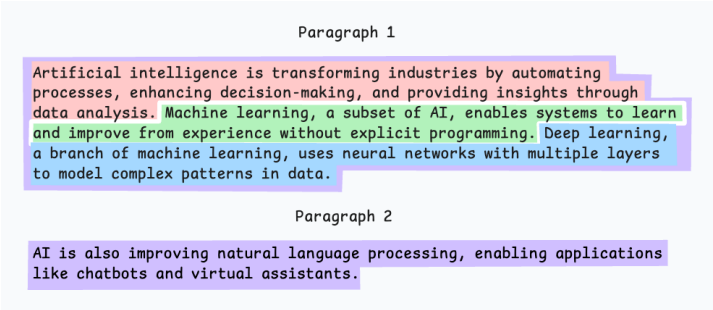


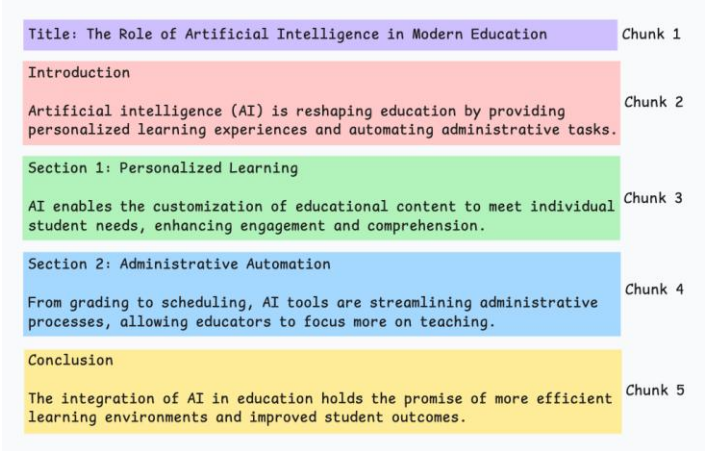
  - LLM-based chunking





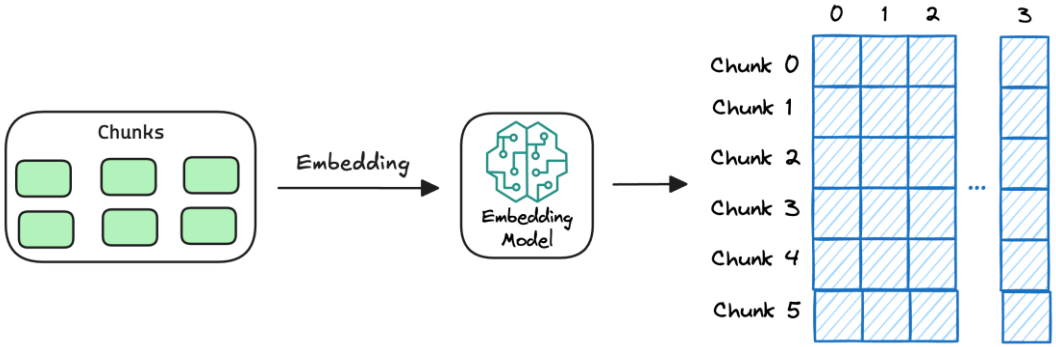


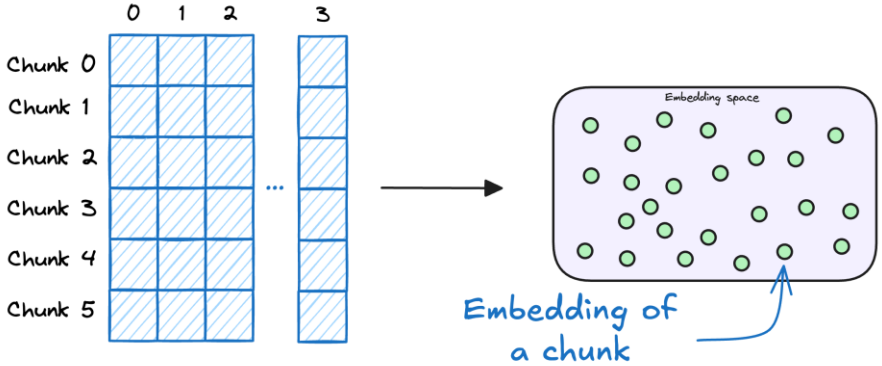




RAG -workflow

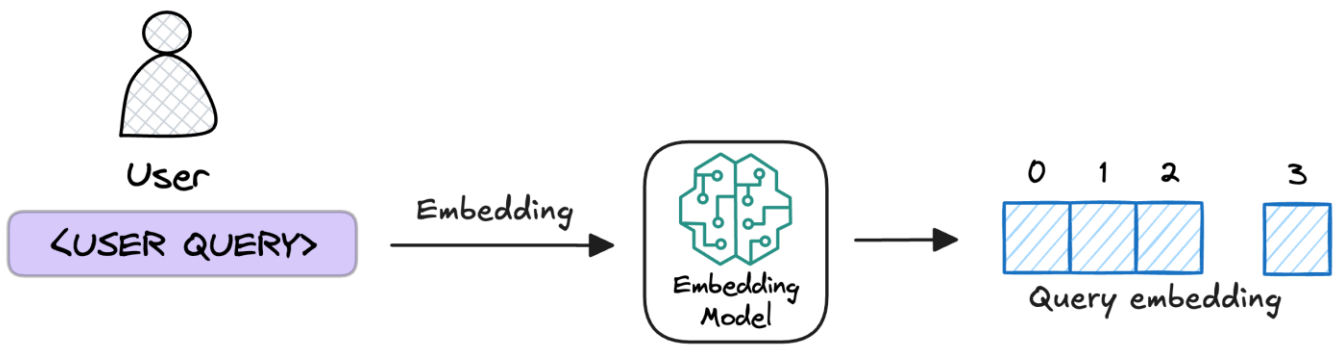
- Indexing phase
  - Chunking
  - Generate embeddings
    - Glove
    - Word2Vec
    - FastText
    - BERT
    - …
  - Store embeddings in a vector database



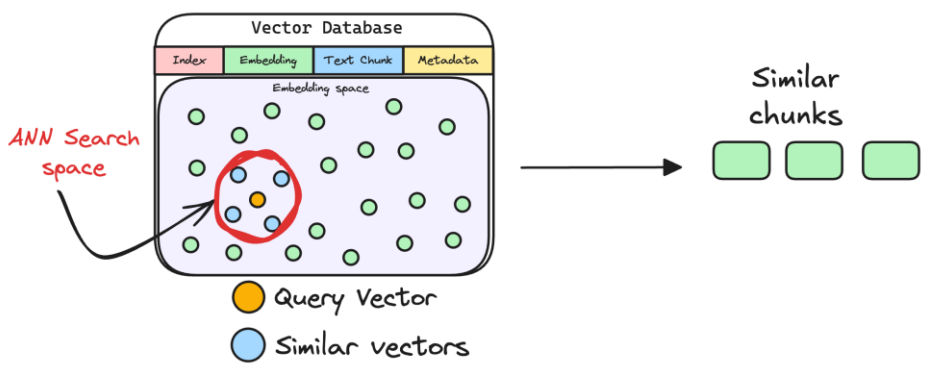


RAG -workflow

- Indexing phase
- Retrieval phase
  - User input query and embedding the query



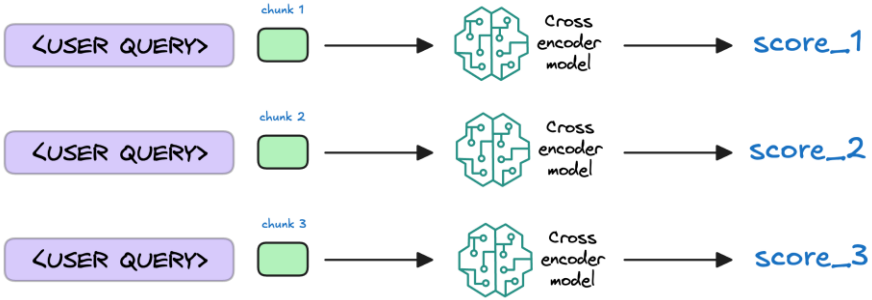
  - Retrieve similar chunks



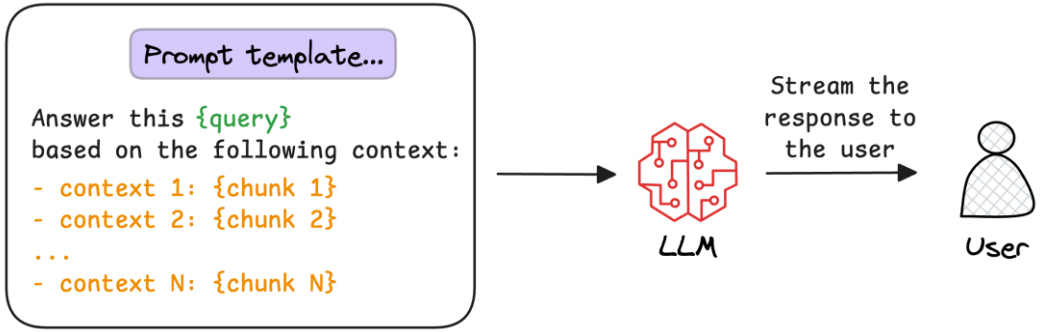
$$similarity(A,B) = cos(\theta) = \frac{A \cdot B}{||A||\,||B||} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2 \sum_{i=1}^{n} B_i^2}}.$$

RAG -workflow

- Indexing phase
- Retrieval phase
  - User input query and embedding the query
  - Retrieve similar chunks
  - Re-rank the chunks

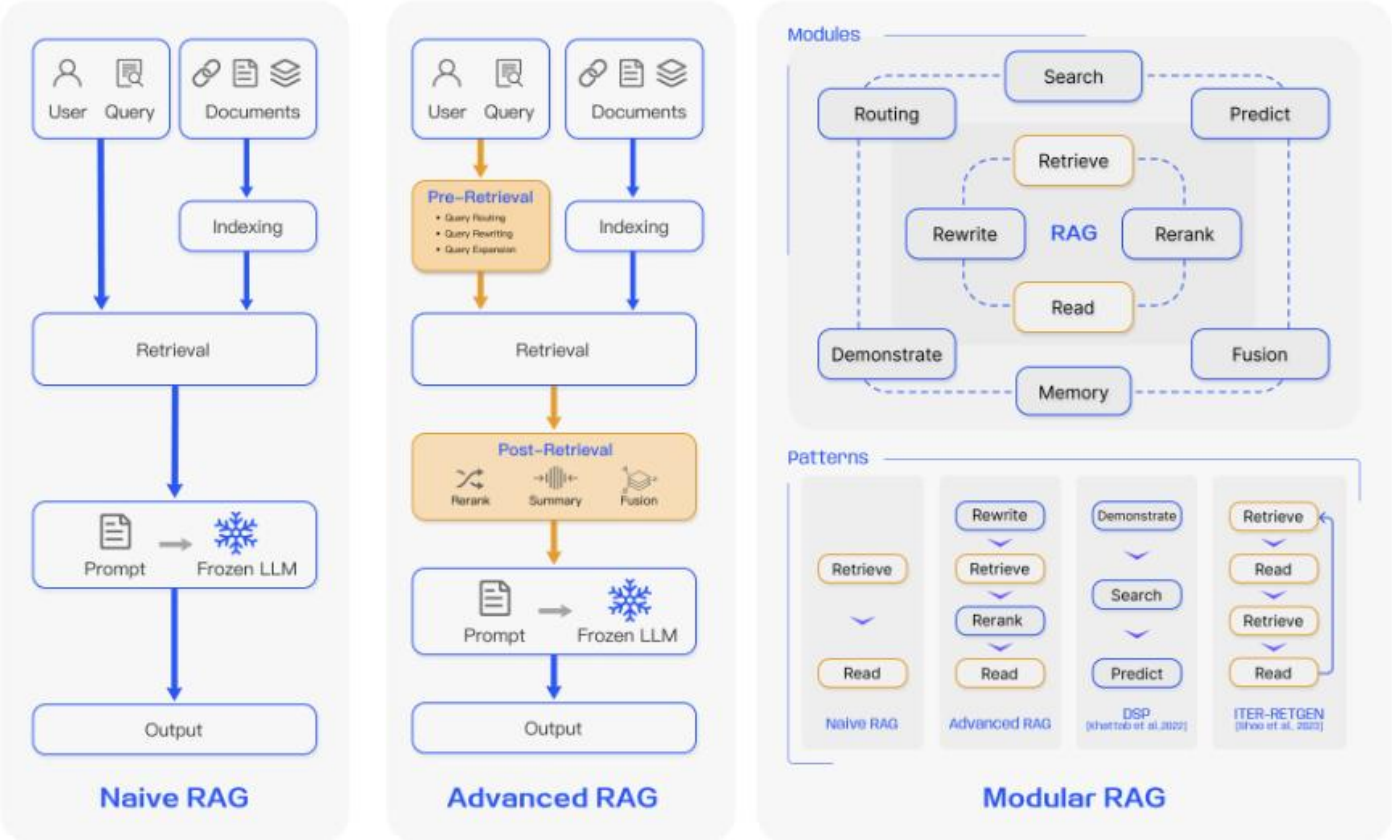


  - Generate the final response



RAG

- Paradigmes



Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., ... & Wang, H. (2023). Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, *2*(1), 32 - https://arxiv.org/pdf/2312.10997

RAG – evaluation

- Metrics
  - Context Relevance – retrieval component
    - Measures how relevant the retrieved chunks are to the user's query
    - Context precision (how many are relevant to the prompt, instead of being noise)
    - Context recall (how many of the most relevant documents to answer the prompt have been retrieved)
  - Answer Relevance – generation component
    - Assesses the LLM's ability to generate useful answers
    - Faithfulness (how factually accurate is the generated answer - response vs. retrieved documents)
    - Answer relevancy (how relevant is the generated answer to the question)
      - human judgment or via automated semantic similarity scoring
  - Grounded-ness
    - Determines the relation of LLM's responses to the retrieved information.
  - Ground Truth
    - Measures alignment between the LLM's response and a predefined correct answer.

- Frameworks
  - DeepEval
  - MLflow LLM evaluate
  - RAGAs

RAG

- Improvements
  - Long context management in RAG
    - How to incorporate as much relevant retrieved knowledge as possible in the initial user query before passing it to the LLM?
      - Chunking strategy - Splits documents into smaller, coherent chunks to preserve context while reducing redundancy and staying within LLM limits.
      - Selective retrieval - Filters large sets of relevant documents to retrieve only the most pertinent parts, minimizing extraneous information
      - Targeted retrieval - Optimizes retrieval for specific query intents using specialized retrievers, adding domain-specific criteria to refine results
      - Context summarization - Uses extractive or abstractive summarization techniques to condense large amounts of retrieved content, ensuring essential information is passed to the LLM.
  - Re-ranking
    - Cross-encoder vs. Bi-encoder
  - Query rewriting